

C 프로그래밍 기초

C Programming for Mechanical Engineers

Week 4-2

반복문 심화: for 확장 · 중첩 반복 · break · continue
#define · Arduino 과제 A3

서강대학교 기계공학과

C week4_2.c

```
1  #define N 3
2
3  int i, j;
4  for (i = 0; i < N; i++)
5  {
6      for (j = 0; j < 3; j++)
7      {
8          if (j == 1) continue;
9          printf("%d %d\n", i, j);
10     }
11 }
```

반복문을 실제로 쓰기 위한 나머지 도구들

- **지난 시간 복습:** 반복의 네 단계(초기화 → 조건 → 몸체 → 갱신), while / do ...while / for
- **for의 확장:** 여러 변수를 한 번에, 복합 조건, 비워 둘 수 있는 자리
- **중첩 반복:** 반복문 안의 반복문. 표, 격자, 모든 조합을 다룰 때
- **break:** 반복을 즉시 끝냄 / **continue:** 이번 바퀴만 건너뛰
- **#define:** 숫자에 이름을 붙여 프로그램을 읽기 쉽고 고치기 쉽게

도구	언제 쓰나
for (a, b; ...)	제어 변수가 둘 이상일 때
for 안의 for	행과 열, 모든 짝을 검사할 때
break	찾았거나 오류가 나서 더 돌 필요가 없을 때
continue	이번 값만 빼고 계속 돌 때
#define	반복 횟수, 한계값 같은 상수

오늘 배우는 것은 전부 지난 시간 for와 while 위에 얹는 것입니다. 형식이 헛갈리면 지난 시간 19쪽(세 반복문 비교)을 다시 보세요.

for 확장 ①: 초기화와 증감에 변수를 여러 개 쓸 수 있습니다

```

1  p = 1;
2  for (n = 0; n < 5; ++n)
3      ...
4
5  /* 아래처럼 한 줄로 */
6  for (p = 1, n = 0; n < 5; ++n)
7      ...
8
9  /* 증감부에도 둘 이상 */
10 for (n = 1, m = 50; n <= m; n++, m--)
11     ...

```

- 여러 항목은 쉼표(,)로 나눕니다. 세미콜론은 여전히 두 개뿐입니다.
- 오른쪽: n과 p를 한 줄에서 초기화하고, 몸체에서 p를 두 배씩 키웁니다.

편리하지만 세 자리가 길어지면 읽기 어렵습니다. 반복과 직접 관계없는 초기화는 for 앞에 따로 쓰세요.

C multi_init.c

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int n, p;
6
7      for (p = 1, n = 0; n < 5; ++n)
8      {
9          p = p * 2;
10         printf("n = %d, p = %d\n", n, p);
11     }
12     return 0;
13 }

```

TERMINAL

```

n = 0, p = 2
n = 1, p = 4
n = 2, p = 8
n = 3, p = 16
n = 4, p = 32

```

for 확장 ②: 조건식은 제어 변수 이외의 것도 검사할 수 있습니다

C sentinel.c

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int i, sum = 0;
6
7      for (i = 1; i < 20 && sum < 100; ++i)
8      {
9          sum = sum + i;
10         printf("%d %d\n", i, sum);
11     }
12     return 0;
13 }
```

TERMINAL

```
...
12 78
13 91
14 105
```

- 조건 `i < 20 && sum < 100`은 두 조건이 모두 참인 동안만 반복합니다.
- `i`는 계수기(counter), `sum`은 보조(sentinel) 변수라고 부릅니다. 보조는 몸체 안에서 계산됩니다.
- `sum`이 105가 되어 조건이 거짓이 되면 `i`가 20에 이르기 전에 끝납니다. 14번 돌았습니다.
- 조건에는 지난주 배운 관계·논리 연산자 무엇이든 쓸 수 있습니다.

"횟수 상한 + 다른 조건"의 조합은 실무에서 아주 흔합니다. 예: 최대 100번까지 센서를 읽되, 값이 안정되면 바로 멈추기.

for 확장 ③: 세 자리 중 일부를 비워 둘 수 있습니다

C empty_for.c

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int m = 5;
6
7      for (; m <= 25;)
8      {
9          printf("%d ", m);
10         m = m + 5;
11     }
12     printf("\n");
13     return 0;
14 }
```

TERMINAL

5 10 15 20 25

- 초기화를 for 앞에서 했고 갱신을 몸체 안에서 하므로, 그 자리를 비웠습니다. 세미콜론은 남겨야 합니다.
- **for (;;)** 처럼 세 자리를 모두 비우면 조건이 없어 무한 루프입니다. while (1)과 같습니다. 안에서 break로 나와야 합니다.
- 비어 있는 for는 사실상 while과 같습니다. 그럴 바에는 while로 쓰는 것이 읽기 쉽습니다.

```

1  for (;;)          /* 무한 루프. break로 탈출 */
2  {
3      ...
4      if (done) break;
5  }
```

지난 시간 실수 ②: for (...); 처럼 괄호 뒤 세미콜론은 몸체가 빈 반복입니다. 교재는 시간 지연용으로 소개하지만 이 수업에서는 쓰지 않습니다.

중첩 반복: 반복문 안에 반복문을 넣습니다

```

1 for (i = 1; i <= 4; i++)    /* 바깥 */
2 {
3     for (j = 1; j <= i; j++) /* 안쪽 */
4     {
5         ...
6     }
7 }

```

- 바깥이 한 바퀴 돌 때마다 안쪽은 처음부터 끝까지 다 돌립니다.
- 안쪽 조건에 바깥 변수 i 를 쓸 수 있습니다 ($j \leq i$).
- 들여쓰기와 중괄호로 어느 문장이 어느 반복 안에 있는지 드러내세요.

C stars.c

```

1 #include <stdio.h>
2 int main(void)
3 {
4     int i, j;
5     for (i = 1; i <= 4; i++)
6     {
7         for (j = 1; j <= i; j++)
8         {
9             printf("*");
10        }
11        printf("\n");
12    }
13    return 0;
14 }

```

TERMINAL

```

*
**
***
****

```

i	j	출력
1	1	*
2	1, 2	**
3	1, 2, 3	***
4	1, 2, 3, 4	****

안쪽 `printf("*")`는 모두 $1+2+3+4 = 10$ 번 실행되고, `printf("\n")`은 바깥 반복에 있으므로 4번 실행됩니다. 이 둘을 구별하는 것이 중첩 반복 읽기의 핵심입니다.

예제: 행과 열로 표를 찍을 때 중첩 for가 자연스럽습니다

C multab.c

```
1  #include <stdio.h>
2  int main(void)
3  {
4      int row, col;
5      for (row = 1; row <= 3; row++)
6      {
7          for (col = 1; col <= 9; col++)
8          {
9              printf("%3d", row * col);
10             }
11             printf("\n");
12         }
13     return 0;
14 }
```

TERMINAL

```
1  2  3  4  5  6  7  8  9
2  4  6  8 10 12 14 16 18
3  6  9 12 15 18 21 24 27
```

- 바깥 row가 행, 안쪽 col이 열입니다. 안쪽이 한 줄을 다 찍고 나면 줄바꿈.
- %3d로 폭을 고정해야 열이 맞습니다 (3주차 출력 폭).
- 몸체 printf는 $3 \times 9 = 27$ 번 실행됩니다. 중첩 반복의 실행 횟수는 곱셈입니다.
- 교재 원본은 do...while 두 개를 겹쳐 12×10 표를 찍습니다. 어느 반복문이든 중첩할 수 있습니다.

행 3개, 열 9개를 각각 #define ROWMAX, COLMAX로 이름 붙이면 표 크기를 한 곳에서 바꿀 수 있습니다. 뒤에서 배웁니다.

실습 1: 구구단 2단부터 9단까지 출력합니다

	할 일
1	바깥 for로 단(dan)을 2부터 9까지 돌립니다.
2	안쪽 for로 곱하는 수(n)를 1부터 9까지 돌리며 <code>printf("%d x %d = %2d\n", dan, n, dan * n)</code> 을 출력합니다.
3	한 단이 끝날 때마다 빈 줄을 하나 출력합니다.

- 파일 이름은 gugudan.c로 저장합니다.
- 빈 줄 출력 `printf("\n")`이 바깥 반복 안, 안쪽 반복 밖에 있어야 합니다. 위치를 틀리면 매 줄마다 빈 줄이 생깁니다.
- 완성한 사람: 한 줄에 2단부터 9단까지 가로로 나란히 찍히게 바뀌 보세요 (바깥과 안쪽을 바꾸면 됩니다).

예상 실행 화면 (앞부분)

```

2 x 1 =  2
2 x 2 =  4
...
2 x 9 = 18

3 x 1 =  3
...
```

표를 찍는 문제는 "행이 무엇이고 열이 무엇인가"만 정하면 중첩 for가 바로 나옵니다.

break: 반복을 즉시 끝내고 반복문 다음 문장으로 갑니다

```
1 while (test-condition)
2 {
3     ...
4     if (condition)
5         break;
6     ...
7 }
8 statement-x; /* break 후 여기로 */
```

- break를 만나면 남은 몸체를 건너뛰고 반복 자체를 끝냅니다.
- while, do...while, for 어디서든 쓸 수 있습니다. switch에서 본 그 break입니다.
- 중첩 반복에서는 break를 감싼 가장 안쪽 반복 하나만 빠져나옵니다.

오른쪽은 지난 시간 sum5000.c를 원본 강의처럼 while (1) + break로 쓴 것입니다. 결과는 같습니다.

C break5000.c

```
1 #include <stdio.h>
2 int main(void)
3 {
4     int sum = 0, num = 0;
5     while (1)
6     {
7         sum += num;
8         if (sum > 5000)
9             break;
10        num++;
11    }
12    printf("num: %d\n", num);
13    printf("sum: %d\n", sum);
14    return 0;
15 }
```

TERMINAL

```
num: 100
sum: 5050
```

예제: 중첩 반복과 break로 30 이하의 소수를 찾습니다

C prime.c

```
1  #include <stdio.h>
2  int main(void)
3  {
4      int i, j;
5      for (i = 2; i <= 30; i++)
6      {
7          for (j = 2; j < i; j++)
8          {
9              if (i % j == 0)
10                 break;
11            }
12            if (i == j)
13                printf("%d ", i);
14        }
15        printf("\n");
16        return 0;
17    }
```

TERMINAL

2 3 5 7 11 13 17 19 23 29

- 바깥 i가 후보, 안쪽 j가 2부터 i-1까지 나누어 봅니다.
- 나누어떨어지면($i \% j == 0$) 소수가 아니므로 안쪽 반복을 break로 끝냅니다.
- 끝까지 안 나누어졌으면 안쪽이 $j == i$ 로 자연스럽게 끝나고, break로 나왔으면 $j < i$ 입니다. 그래서 $i == j$ 로 소수를 판별합니다.
- break는 안쪽 for만 빠져나옵니다. 바깥 for는 다음 i로 계속 갑니다.

반복이 끝난 뒤 제어 변수의 값으로 "중간에 나왔는지, 끝까지 갔는지"를 알아내는 관용구입니다. 지난 시간 digits.c의 $x = 10$ 을 떠올리세요.

continue: 이번 바퀴의 남은 부분을 건너뛰고 다음 바퀴로 갑니다

```

1 for (init; test; increment)
2 {
3     ...
4     if (condition)
5         continue; /* 아래 건너뛸 */
6     ...           /* 다음 바퀴로 */
7 }

```

- continue를 만나면 그 아래 문장들을 건너뛰고 다음 반복으로 넘어갑니다. 반복은 계속됩니다.
- for에서는 increment를 실행한 뒤 조건 검사로, while과 do...while에서는 바로 조건 검사로 갑니다.
- break는 "그만", continue는 "이번 것은 빼고 계속"입니다.

오른쪽: 2의 배수이거나 3의 배수인 수는 출력을 건너뛰고 1~19의 나머지만 찍습니다.

C skip.c

```

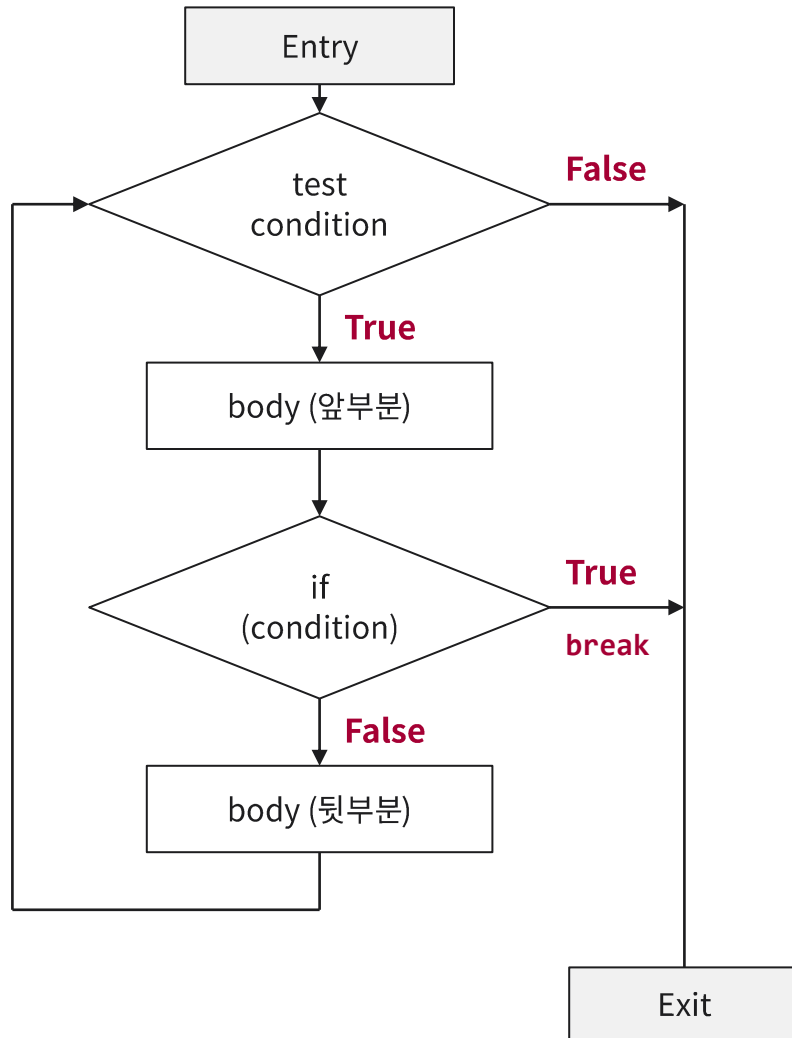
1 #include <stdio.h>
2 int main(void)
3 {
4     int num;
5     printf("start! ");
6     for (num = 1; num < 20; num++)
7     {
8         if (num % 2 == 0 || num % 3 == 0)
9             continue;
10        printf("%d ", num);
11    }
12    printf("end!\n");
13    return 0;
14 }

```

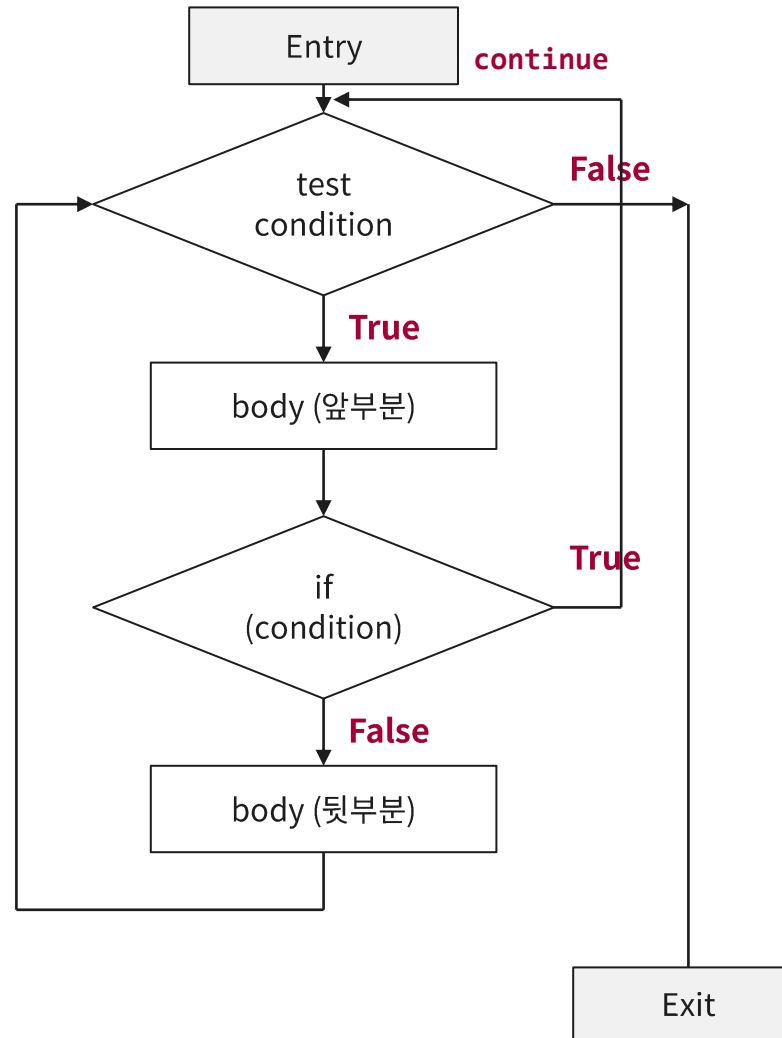
TERMINAL

start! 1 5 7 11 13 17 19 end!

break와 continue의 흐름: 어디로 점프하는가



(a) break: 반복 밖으로 (교재 그림 7.9)



(b) continue: 조건 검사로 (교재 그림 7.13, while의 경우)

	간다
break	반복 밖 Exit
continue	조건 검사 (for는 증감 먼저)

둘 다 if와 함께 씁니다. if 없이 쓰면 첫 바퀴에서 무조건 점프하므로 의미가 없습니다.

예제: 센서 값 N개를 읽되, 음수면 중단하고 100 초과는 무시합니다

C sensor_max.c

```
1  #include <stdio.h>
2  #define N 6
3  int main(void)
4  {
5      int i;
6      double v, max = -1.0;
7      for (i = 1; i <= N; i++)
8      {
9          printf("Sensor %d: ", i);
10         scanf("%lf", &v);
11         if (v < 0.0)
12         {
13             printf("Negative value, stop.\n");
14             break;
15         }
16         if (v > 100.0)
17             continue;
18         if (v > max)
19             max = v;
20     }
21     printf("max = %.1f\n", max);
22     return 0;
23 }
```

TERMINAL

```
Sensor 1: 40.5
Sensor 2: 120
Sensor 3: 55.2
Sensor 4: 61.0
Sensor 5: -1
Negative value, stop.
max = 61.0
```

- 음수는 센서 고장으로 보고 break: 더 읽지 않고 반복을 끝냅니다.
- 100 초과는 튀는 값으로 보고 continue: 이 값만 빼고 다음 센서로 갑니다.
- #define N 6이 반복 횟수입니다. 다음 슬라이드에서 배웁니다.

실제 로봇 코드에서 센서 값 검사는 정확히 이 형태입니다. 치명적이면 break, 그 값만 이상하면 continue.

#define: 컴파일 전에 이름을 다른 코드로 치환합니다

```
#define identifier string
```

- 전처리기(preprocessor)가 컴파일 전에 identifier를 string으로 모두 바꿔치기합니다 (매크로 치환).
- #include처럼 #으로 시작하는 줄이고, 끝에 세미콜론을 쓰지 않습니다.
- 보통 파일 위쪽에 두고, 이름은 대문자로 씁니다 (MAX_TEMP, N_SAMPLES).
- 큰따옴표 안의 문자열 속 글자는 바뀌지 않습니다.

```
1 #define M 5
2
3 total = M * value;
4 printf("M = %d\n", M);
```

전처리 후 컴파일러가 보는 코드:

```
1 total = 5 * value;
2 printf("M = %d\n", 5);
3 /* 문자열 안의 M은 그대로 */
```

프로그램 여기저기에 3이나 80.0 같은 숫자가 흩어져 있으면, 나중에 값을 바꿀 때 하나라도 놓치기 쉽습니다. #define으로 한 곳에 모으면 한 줄만 고치면 됩니다. 이런 흩어진 숫자를 매직 넘버라고 부르며 피해야 할 습관입니다.

예제: 반복 횟수와 한계값을 #define으로 뽑아냅니다

C define.c

```
1 #include <stdio.h>
2 #define MAX_TEMP 80.0
3 #define N_SAMPLES 5
4 int main(void)
5 {
6     int i;
7     double temp, sum = 0.0;
8     for (i = 1; i <= N_SAMPLES; i++)
9     {
10         printf("Sample %d: ", i);
11         scanf("%lf", &temp);
12         if (temp > MAX_TEMP)
13             printf(" over %.1f!\n", MAX_TEMP);
14         sum += temp;
15     }
16     printf("average = %.1f\n", sum / N_SAMPLES);
17     return 0;
18 }
```

TERMINAL

```
Sample 1: 70
Sample 2: 85
    over 80.0!
... (5개 입력)
average = 76.0
```

- MAX_TEMP와 N_SAMPLES가 세 곳에 쓰였습니다. 샘플 수를 10으로 바꾸려면 위의 한 줄만 고칩니다.
- #define 상수는 변수가 아닙니다. 값을 대입할 수 없고 메모리도 차지하지 않습니다. 그냥 글자 바꿔치기입니다.
- printf의 인수 자리에 MAX_TEMP를 넣은 것도 치환되어 80.0이 됩니다.

Arduino 코드에서도 핀 번호와 시간을 #define LED_PIN 13, #define ON_TIME 1000처럼 쓰는 것이 관례입니다. A3에서 써 보세요.

주의: 식을 #define할 때는 괄호로 감싸세요

C define_bug.c

```
1 #include <stdio.h>
2 #define D 45-22
3 #define A (78+32)
4
5 int main(void)
6 {
7     printf("%d\n", D / A);
8     printf("%d\n", (45-22) / (78+32));
9     return 0;
10 }
```

TERMINAL

```
45
0
```

- D / A 는 글자 그대로 $45-22 / (78+32)$ 로 바뀝니다. 나눗셈이 먼저 계산되어 $45 - 0 = 45$ 가 됩니다.
- 의도한 $(45-22) / (78+32) = 23 / 110$ 은 정수 나눗셈이라 0입니다.
- `#define D (45-22)`처럼 식 전체를 괄호로 감싸면 치환 뒤에도 순서가 보존됩니다.
- 전처리기는 수식을 이해하지 않습니다. 글자만 바꿉니다. 그래서 괄호가 필요합니다.

#define으로 식을 정의할 때는 무조건 괄호. 이 수업에서 #define은 주로 단순한 숫자 상수에만 쓸 것이므로, 규칙만 기억하면 됩니다.

실습 2: 센서 값 N개의 평균, 단 음수면 중단하고 100 초과는 제외합니다

	할 일
1	#define N 5로 샘플 수를 정하고, for로 N번 scanf("%lf")를 반복합니다.
2	음수가 들어오면 break, 100을 넘으면 continue. 정상 값만 sum에 더하고 count를 1 늘립니다.
3	반복이 끝난 뒤 count가 0보다 크면 평균을 %.1f로, 아니면 "no valid sample"을 출력합니다.

- 파일 이름은 sensor_avg.c로 저장합니다.
- 평균은 sum / N 이 아니라 $\text{sum} / \text{count}$ 입니다. 제외된 값과 중단된 뒤의 값은 세지 않습니다.
- count가 0일 때 나누면 안 됩니다. 반복 뒤의 if...else가 그것을 막습니다.
- 완성한 사람: 최댓값과 최솟값도 함께 출력해 보세요 (앞의 sensor_max.c 참고).

예상 실행 화면

```
Sample 1: 40.5
Sample 2: 120
Sample 3: 55.5
Sample 4: -1
average of 2 samples = 48.0
```

평균, 최대, 최소를 반복 한 번에 구하는 이 형태는 6주차 배열에서 그대로 다시 씁니다.

오늘 정리와 다음 시간

- for 확장: 쉼표로 여러 변수, 조건에 &&로 보초 변수, 자리를 비우면 while과 같음. for (;;)는 무한 루프.
- 중첩 반복: 바깥 한 바퀴마다 안쪽 전부. 실행 횟수는 곱. 어느 문장이 어느 반복에 속하는지 들여쓰기로.
- break는 반복 밖으로(안쪽 하나만), continue는 다음 바퀴로(for는 증감 먼저). 항상 if와 함께.
- 반복 뒤 제어 변수 값으로 중간에 나왔는지 알 수 있습니다 (prime.c의 `i == j`).
- #define: 매직 넘버에 이름 붙이기. 세미콜론 없음, 식은 괄호로.

다음 시간 (5-1)	내용
함수	프로그램을 기능 단위로 나누기. 정의, 호출, 인수, 반환값
왜 함수인가	같은 코드를 두 번 쓰지 않기, 읽기 쉽게 하기
Arduino	setup()과 loop()도 함수입니다

다음 슬라이드가 이번 주 Arduino 과제 A3입니다. 오늘 배운 for와 #define을 실제 보드에서 써 봅니다.

4주차 개별 프로젝트: 신호등 한 주기를 for로 3회 반복



▶ 영상 열기 (YouTube)

<https://www.youtube.com/watch?v=Uah9GYkPYT0>

No_3 초보용 코딩 아두이노 신호등 만들기 (SHYANG)

보충: for 문 설명 영상 youtu.be/V0Cakd43csQ · 선택: while 버전 youtu.be/Lr32cpvTlPg

영상의 신호등 예제를 완성한 뒤, 아래처럼 확장합니다.

빨강 → 초록 → 노랑 한 주기를 for로 3회 반복 후 정지
반복 횟수, 핀 번호, 시간은 #define으로 이름 붙이기
3회 뒤 노랑을 0.2초 간격 5번 깜빡이고 모두 끄기
loop()의 무한 반복 금지. 3회 후 반드시 멈춰야 함

제출	내용
동작영상	3회 반복 후 노랑 깜빡임, 정지까지 30초 내외 영상 (이름, 학번을 쓴 메모를 함께 촬영)
Arduino 코드	실제로 보드에 업로드한 스케치 파일

Arduino 사용법은 수업에서 설명하지 않습니다. LED 저항은 150 Ω 대신
키트의 330 Ω. **CyberCampus 과제 게시판에 제출**